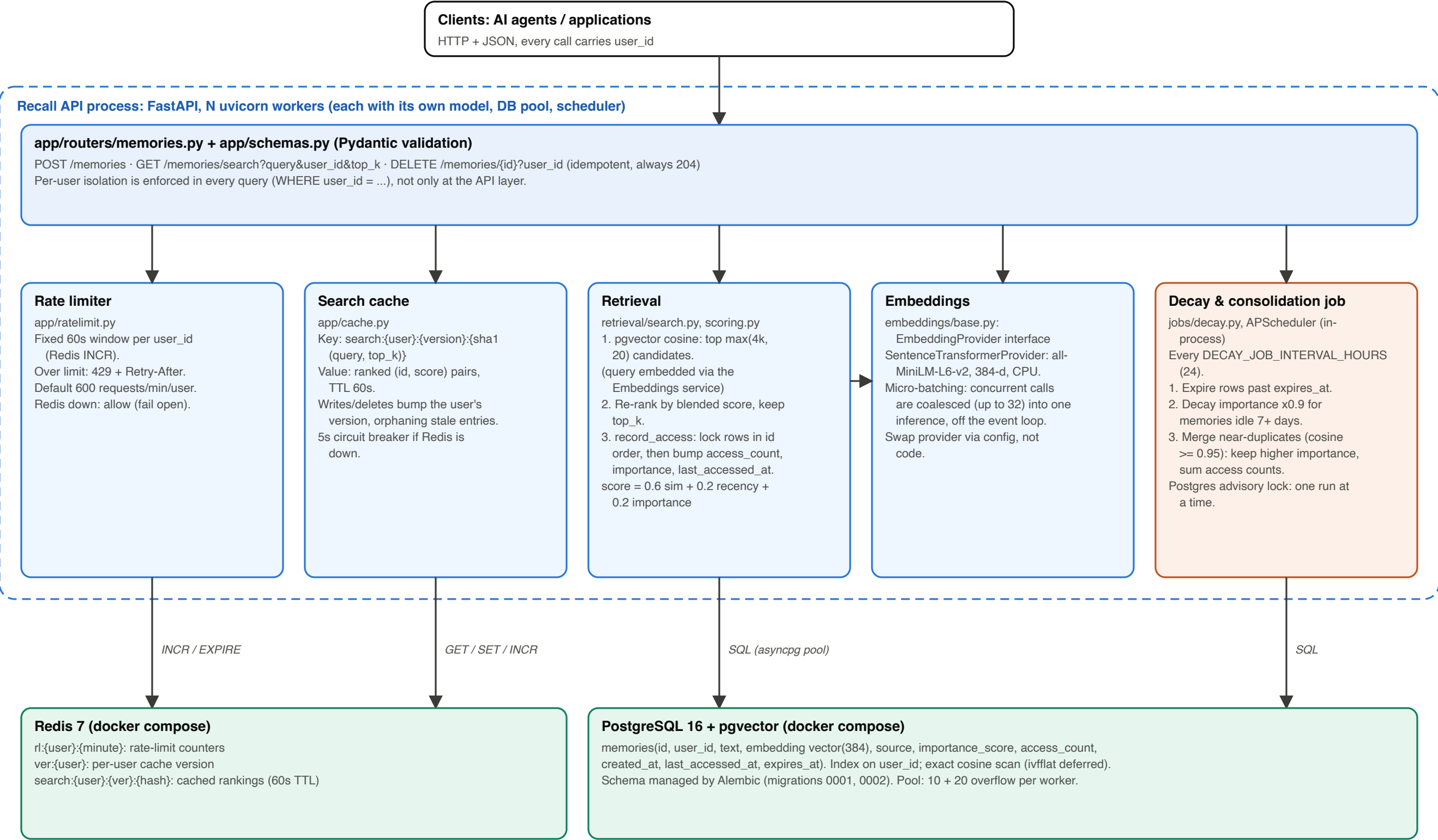


Recall: system architecture

Persistent memory service for AI agents. Redis is optional at runtime (cache and rate limiter fail open).



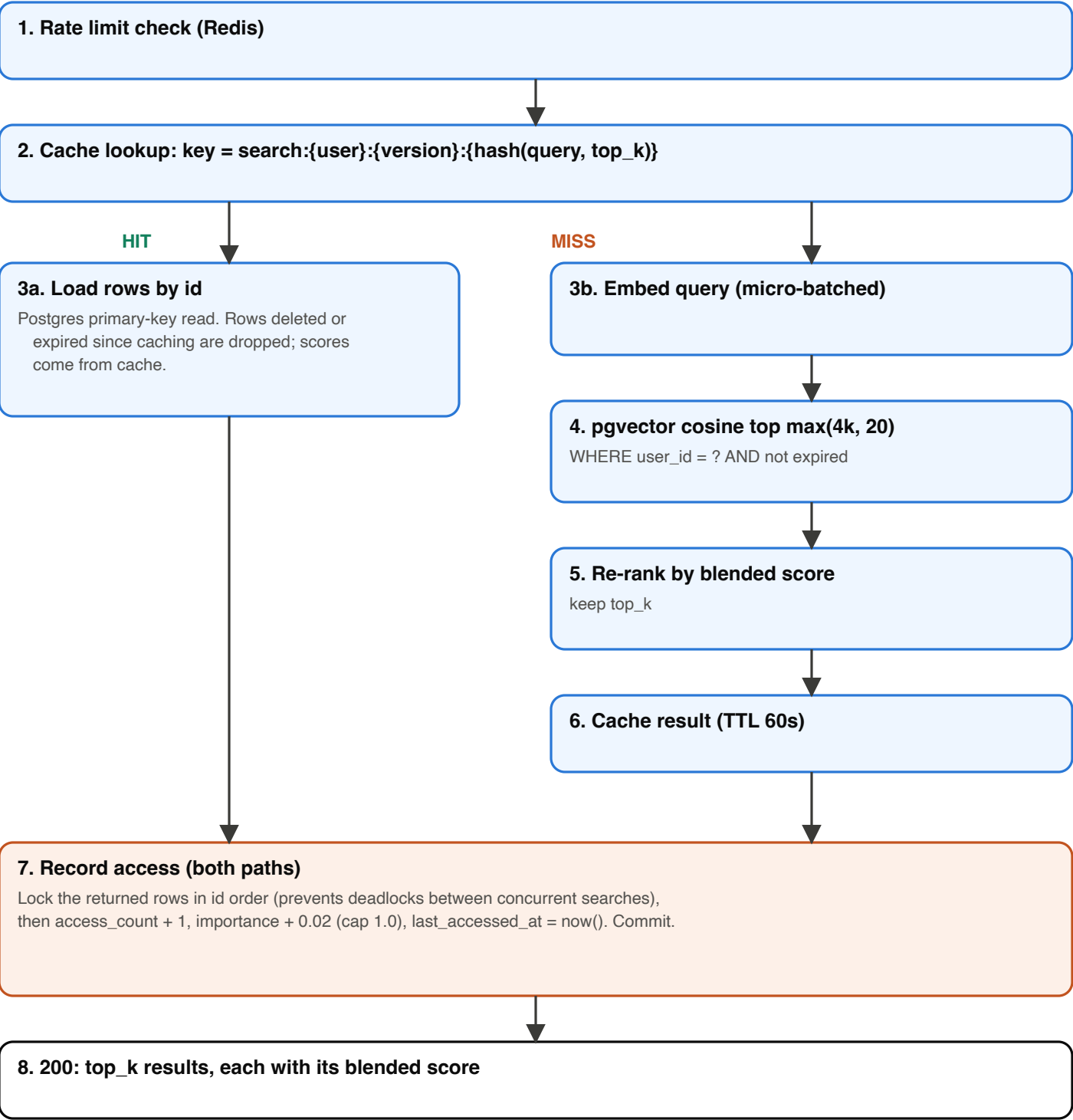
Tooling (not part of the running service)

scripts/loadtest.py: concurrent writes/searches over HTTP · pytest suite against real Postgres + Redis
scripts/simulate.py (in-process app + DB, simulated days) -> data/sim_*.json -> scripts/report.py -> docs/usage_report.html

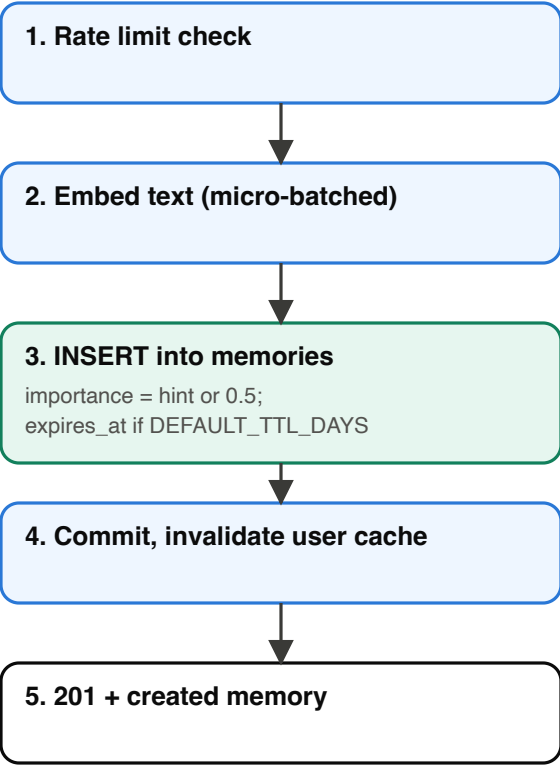
Recall: request flows, background job, scoring

Arrows show the order of steps. Every endpoint starts with the per-user rate limit.

GET /memories/search



POST /memories



Invalidation = INCR ver:{user}; that user's cached searches are orphaned and expire via TTL.

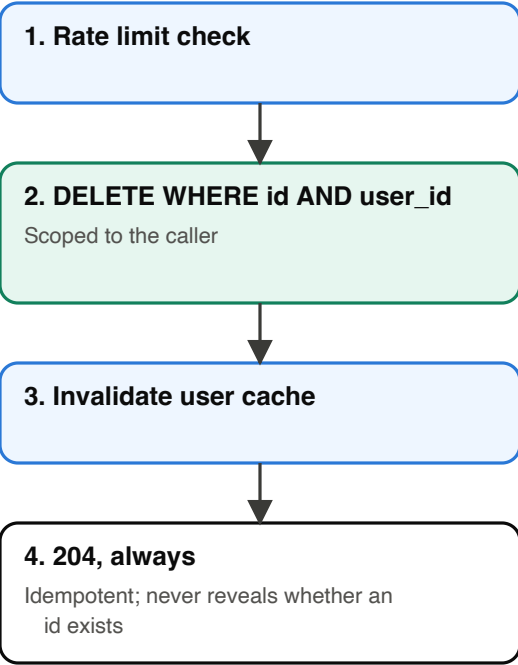
Decay job (background, every 24h)

Runs in one transaction, guarded by a Postgres advisory lock

1. Expire: delete rows with expires_at <= now().
2. Decay: importance x 0.9 (floor 0.05) if not accessed in 7 days; never-accessed rows count from created_at.
3. Consolidate, per user: group pairs with cosine >= 0.95, keep the higher-importance memory, sum access counts, delete the rest, log each merge.

Counterweight to the access bump: searches raise importance, idleness lowers it.

DELETE /memories/{id}?user_id



Scoring (docs/SCORING.md)

final_score = 0.6 x similarity + 0.2 x recency + 0.2 x importance
similarity = cosine normalized to [0, 1]; recency = exp(-age_days / 14) from created_at; importance = stored importance_score (access bumps up, decay job down).
Weights are named constants in app/retrieval/scoring.py, overridable from .env, and must sum to 1.0.